



GPU cluster dynamics: insights from Alibaba's 2023 trace release

Ahmad Siavashi¹ · Mahmoud Momtazpour¹

Received: 4 April 2024 / Accepted: 2 October 2024 / Published online: 20 November 2024

© The Author(s), under exclusive licence to Springer-Verlag GmbH Austria, part of Springer Nature 2024

Abstract

In this paper, we present a comprehensive analysis of GPU cluster traces from Alibaba, released in 2023, focusing on understanding the detailed settings of nodes and pods and the important numbers related to them. By examining the configurations of 1,523 nodes, predominantly GPU-based, we identify a wide array of configurations and GPU models, highlighting a balanced interplay between CPU cores and RAM across all nodes while also noting a decoupling between CPU/RAM and GPU-specific metrics in GPU nodes, indicating a nuanced approach to workload placement. Our investigation extends to an analysis of 8,152 pods, revealing diverse configurations and a significant prevalence of latency-sensitive pods, indicative of a cluster geared towards timely resource provisioning. The presence of *Failed* and *Pending* pods, however, underscores the complexity and challenges inherent in scheduling efficiency and resource allocation. By delving into both node and pod configurations and metrics, this study sheds light on the operational characteristics and workload patterns within the cluster. The insights gleaned from this analysis are beneficial for researchers, system designers, and operators, providing a nuanced understanding of the cluster's workings. This, in turn, lays the groundwork for future research and optimization efforts, ultimately contributing to the advancement of GPU cluster management and operation.

Keywords Cloud computing · Cluster traces · GPU workloads · Resource allocation

Mathematics Subject Classification 68M20

✉ Mahmoud Momtazpour
momtazpour@aut.ac.ir

Ahmad Siavashi
siavashi@aut.ac.ir

¹ Department of Computer Engineering, Amirkabir University of Technology, Tehran, Iran

1 Introduction

The rapid evolution of cloud computing has significantly impacted the landscape of computational resource management, particularly with the integration of Graphics Processing Units (GPUs) in cluster environments [1–3]. GPUs provide substantial parallel processing capabilities, making them beneficial for a wide range of computationally intensive applications [4]. As such, understanding the configuration and workload patterns in GPU clusters is imperative for optimizing their performance and efficiency [5, 6].

This paper presents an in-depth analysis of GPU cluster traces released by Alibaba in 2023 [5, 7], focusing on the configurations of nodes and pods within the cluster. The dataset encompasses 1,523 nodes and 8,152 pods, providing a rich source of information for our investigation. In this study, we seek to identify the inherent trends and features of the workloads operating on the cluster and the configurations of the nodes that facilitate them.

The motivation for this study is twofold. Firstly, we aim to provide system designers, operators, and researchers with insights into the configuration patterns and workload characteristics of a large-scale GPU cluster. By dissecting the configurations and workload patterns within the cluster, we can identify prevalent trends and potential areas for improvement, contributing to the optimization of GPU cluster performance. Secondly, this study adds to the broader discourse on resource management in cloud computing environments. By providing empirical evidence and insights based on real-world data, we contribute to the knowledge base that can guide future research and development efforts in this domain. The main contributions of our work are:

- We examine Alibaba’s 2023 GPU cluster trace to explore the configurations of nodes and pods, uncovering the distribution and frequency of these configurations and the correlations between CPU, RAM, and GPU resources.
- We assess scheduling metrics, including creation times, interarrival times, and scheduling delays, to provide insights into workload arrival characteristics and scheduling performance. These insights can guide future improvements in workload scheduling and resource management strategies.
- We analyze various pod configurations and their associated Quality of Service (QoS) classes, revealing workload patterns such as the prevalence of latency-sensitive tasks. We also examine pod phases over time, noting the occurrences of pod failures and pending states. This characterization provides insights into the operational dynamics and resource demands of different workload types.
- We present the distribution of pod durations across different phases and QoS classes to highlight how long various types of pods remain in the cluster, which helps inform decisions on workload scheduling.

The structure of the paper is as follows. First, we discuss previous research and studies pertinent to our topic. Next, we provide an overview of the nodes in the cluster, detailing their configurations, GPU models, and the distribution of

GPU and non-GPU nodes. This is followed by an in-depth analysis of the pods in the cluster, exploring their configurations, QoS classes, lifecycle phases, and scheduling metrics. Through this examination, we uncover key operational patterns, trends, and characteristics of the GPU cluster. Finally, we conclude our paper, suggesting directions for future research and optimization in GPU cluster management.

In summary, this paper presents an analysis of a large-scale GPU cluster, providing a granular view of its configuration and operation. By doing so, we aim to contribute to the ongoing efforts to optimize resource allocation, enhance scheduling efficiency, and improve the overall performance of GPU clusters in cloud computing environments.

2 Related work

The Alibaba Cluster Trace Program has engendered a wealth of research by offering traces from real-world production environments. These traces have been pivotal in decoding the characteristics and workloads of modern internet data centers (IDCs). Over time, six versions of traces have been unveiled, each encapsulating unique insights into different aspects of cluster operations and workloads. The Cluster Trace v2017 has been instrumental in various studies exploring a myriad of themes including hardware resource disaggregation, cloud workload elasticity and plasticity, straggler diagnosis, datacenter workload characterization, imbalance in the cloud, and co-allocation characteristics in internet data centers [8–13].

The Cluster Trace v2018 has been the cornerstone for numerous studies examining resource efficiency, job scheduling, task dependencies, and maximum flow management in shared production clusters, respectively [14–17]. The Cluster Trace GPU v2020 facilitated a profound investigation into workload analysis and scheduling in large-scale heterogeneous GPU clusters, providing a glimpse into MLaaS (Machine Learning as a Service) in real-world settings [6].

In the realm of microservices, the Microservices Trace v2021 served as a foundational dataset for respective studies that scrutinized microservice dependency and performance, and crafted an open-source factory of benchmark microservice applications, both advancing the understanding and benchmarking of microservices [18, 19]. Moving on to Microarchitecture Trace v2022, it played a pivotal role in an exploration of job microarchitectural profiles at a large scale, shedding light on a comprehensive dataset and analysis [20]. The Cluster Trace GPU v2023 has notably been utilized in a study which delved into the scheduling of GPU-sharing workloads, putting a spotlight on the challenges of fragmentation in such environments [5].

The diverse range of research topics and methodologies employed in these studies underscores the richness and the beneficial nature of the traces provided by the Alibaba Cluster Trace Program. Through a multitude of lenses, these works collectively contribute to a deeper understanding of modern data centers and workloads, while also advancing the state-of-the-art in scheduling, resource management, and microservice benchmarking, among others.

3 Trace data

In this section, we delve into the raw data traces and the accessible information, which are available through Alibaba Open Source [7]. The data encompass the configuration of 1,523 nodes within a production heterogeneous GPU cluster, detailing the CPU capacity (initially provided in milli units), main memory (originally in MiB), the number of GPUs, and the GPU model for each node. For improved readability, we have converted the units from milli and MiB to resource fractions and GiB, respectively.

The data set details attributes of 8,152 pods, encompassing attributes such as `cpu_milli`, indicating the requested number of CPUs in milli units; `memory_mib`, specifying the requested main memory in Mebibytes; and `num_gpu`, denoting the requested number of GPUs, with values ranging from 0 to 8. The attribute `gpu_milli` provides detailed GPU requests for GPU-sharing workloads in milli units. The `gpu_spec` attribute outlines the required GPU types. The `pod_phase` attribute indicates the pod's state. In Kubernetes environments [21], a pod is *Pending* when accepted by the cluster but its containers are not yet ready, covering both scheduling delays and container image downloads. The *Running* phase indicates a pod has been assigned to a node, with all containers created and at least one actively running or starting. A pod is *Succeeded* when all its containers have successfully completed their tasks and will not restart. If a pod is in the *Failed* phase, it means all containers have stopped, with at least one having terminated due to failure, either by exiting with a non-zero status or by system termination.

The QoS is denoted by the `qos` attribute. Kubernetes classifies pods into three QoS classes [22]: *Guaranteed*, *Burstable*, and *BestEffort* (BE), based on their resource requests and limits. *Guaranteed* pods have strict resource limits and are the least likely to be evicted, requiring equal memory and CPU limits and requests for all containers. *Burstable* pods have minimum resource guarantees without specific limits, allowing them to utilize additional resources when available and are evicted only after BE pods under node pressure. BE pods have no specific resource requests or limits, making them the first to be evicted during resource shortages. They can use any available resources not allocated to higher-priority pods. The dataset additionally features a *Latency Sensitive* (LS) class, though specifics are not provided as it does not correspond to a predefined QoS class within Kubernetes.

The dataset contains key timestamps reflecting pod lifecycle events. The `creation_time` marks the pod's initial recognition by Kubernetes, highlighting its entry into the cluster. The `scheduled_time` shows when the pod was allocated to a node, indicating its move from pending to ready for execution. Finally, `deletion_time` signals when the system received a deletion request, beginning the pod's removal process. The timestamps are expressed in seconds, and similar to other attributes, units have been converted during analysis for convenience.

The dataset does not encompass details pertaining to the network or topology that interlinks the nodes, nor does it include information about the distribution of

Pods among these nodes. As a result, evaluating the utilization of node resources is unattainable. Furthermore, the dataset omits information regarding the quantity of cores within the CPUs of the nodes as well as their operational frequencies. Moreover, the characteristics of the two GPU models, referred to as G2 and G3, remain undisclosed. Pod resource requirements are not detailed and operational logs such as why a pod is failed is not given.

4 Nodes in the cluster

The cluster comprises 1,523 nodes, with a substantial majority being GPU nodes, which make up 79.65% of the total. Conversely, non-GPU nodes constitute a smaller portion, accounting for 20.35% of the cluster. Figure 1 shows the distribution of node configurations based on CPU cores and RAM, distinguishing GPU nodes (open circles) from non-GPU nodes (cross markers). Marker size reflects the number of nodes with each configuration, with larger markers indicating more common setups. A clustering of GPU nodes around specific CPU and RAM combinations suggests favored configurations for GPU-intensive tasks. Although less frequent, non-GPU nodes also span various CPU and RAM configurations, with preferences for certain combinations. This pattern highlights a focus on GPU acceleration within the cluster, reflecting strategic resource optimization for high-performance computing, while also accommodating diverse workloads. Figure 2 provides a detailed breakdown of the distribution of GPU models within the GPU nodes. Moreover, the configuration of each GPU model is provided in Table 1.

The G2 model, with its 96 CPU cores, 384 GB RAM, and 8 GPUs, is the most prevalent in the cluster, accounting for 549 nodes, suggesting that this high-end GPU configuration might be a standard or preferred setup for many applications or

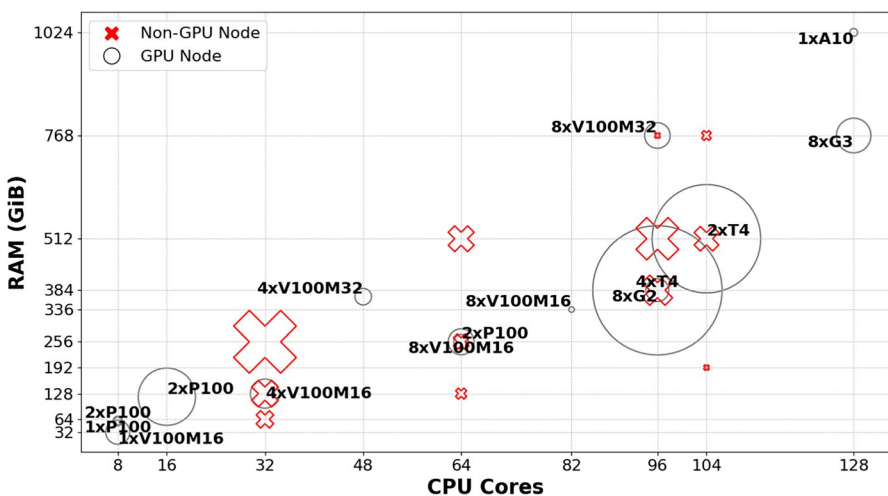


Fig. 1 Node configurations

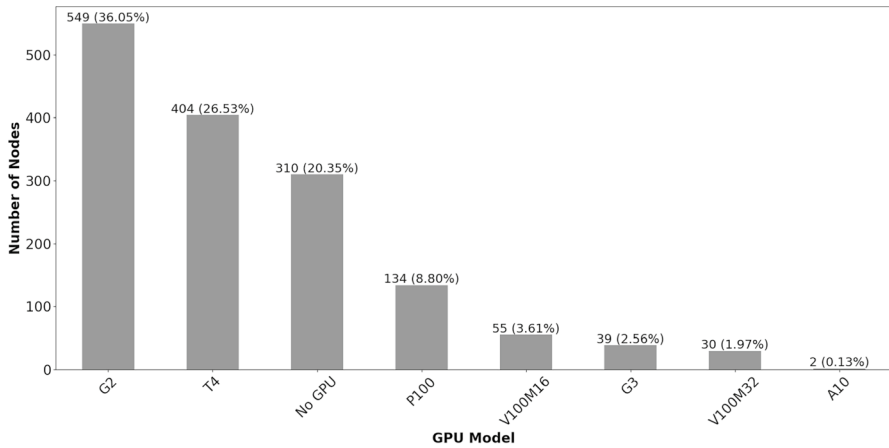


Fig. 2 Distribution of GPU models in the cluster

Table 1 GPU model specifications [23–26]

GPU Model	Memory (GB)	Core Count
P100	16	3584
G3	N/A	N/A
V100M32	32	5120
V100M16	16	5120
G2	N/A	N/A
T4	16	2560
A10	24	9216

tasks within the cluster. The P100 model displays a diverse range of configurations, including configurations with 1 or 2 GPUs. While a majority of P100 nodes (107) operate with 16 CPU cores, 120 GB RAM,¹ and 2 GPUs, there are configurations with 8 CPU cores, 60 GB RAM, and 1 or 2 GPUs, as well as 64 CPU cores with a substantial 256 GB RAM and 2 GPUs, indicating the P100 model's versatility.

The V100M16² and V100M32² models, with 1 to 8 GPUs, exhibit a broader range of RAM configurations, spanning from 32 GB to 768 GB, paired with varying CPU cores. This suggests that these models might be used for tasks that have varying memory and computational requirements. The T4 model, with configurations of 96 and 104 CPU cores, is split between two RAM configurations: 384 GB and 512 GB, and includes 2 or 4 GPUs, indicating two standard setups for this model, catering to different workload requirements. The A10 and G3 models, both operating with a high 128 CPU cores, are associated with larger RAM capacities of 1024 GB

¹ This deviation from the conventional powers-of-two may be attributed to virtualization or deduction of system-reserved memory.

² The V100M16 and V100M32 are 16 GB and 32 GB versions of V100, respectively.

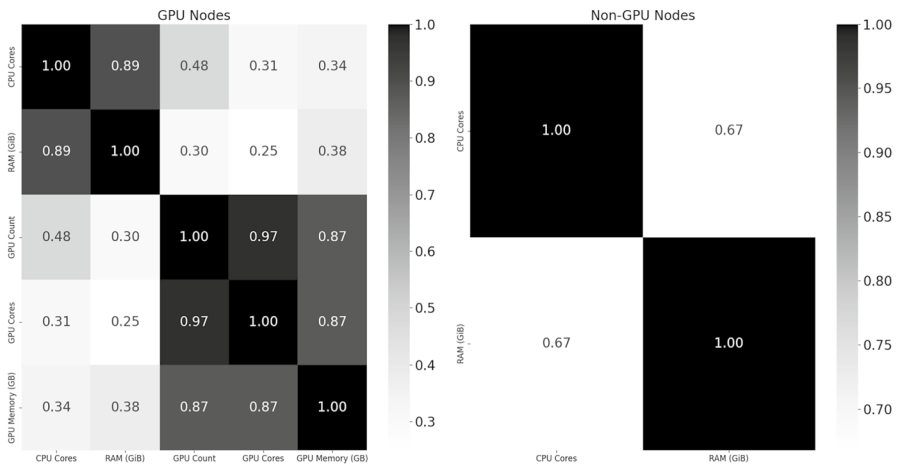


Fig. 3 Correlation matrices of node configurations

and 768 GB, respectively, and have 1 and 8 GPUs respectively. These configurations are likely tailored for highly demanding tasks, requiring substantial computational power and memory.

Non-GPU configurations are also an integral part of the cluster, with configurations ranging from 32 to 104 CPU cores and 64 to 768 GB of RAM. Despite not having the graphical processing power, these setups, totaling 249 nodes, offer substantial computational capabilities. These non-GPU configurations are likely utilized for tasks that are more CPU-bound or do not require GPU acceleration, ensuring efficient resource allocation for a wide array of computational requirements within the cluster.

Analyzing the correlation matrices shown in Fig. 3 provides important insights into the relationships between CPU cores, RAM, GPU cores, GPU memory, and GPU count across different node configurations in the cluster.

Non-GPU Nodes The correlation between CPU cores and RAM (0.67) shows a continued balance in computational and memory resources in nodes without GPUs.

GPU Nodes The correlation between CPU cores and RAM is even stronger in GPU nodes (0.89) than in Non-GPU nodes, suggesting a consistent trend in these configurations. We overlook the relationships among GPU-specific metrics (GPU count, GPU cores, and GPU memory) as they are not relevant to our analysis.³ There is a moderate correlation between CPU cores and GPU count (0.48) and the relationships between CPU cores and GPU cores and memory are weak with values of 0.31 and 0.34, respectively. Similarly, RAM's correlation with GPU count, GPU cores, and GPU memory, at 0.30, 0.25, and 0.38, is also weak.

The consistent and strong correlation between CPU cores and RAM across all configurations indicates that systems are generally balanced in terms of

³ Without G2 and G3 specifications, they are excluded from the calculations.

computational processing and memory capabilities. This balanced configuration ensures that neither the CPU nor the RAM becomes a bottleneck during processing, leading to optimized performance.

However, the generally weak correlation between CPU and RAM with GPU-specific metrics in GPU nodes suggests that GPUs are added based on specific computational needs rather than a proportional increase with CPU and RAM. This decoupling offers flexibility in workload placement. For instance, a task requiring intensive GPU computation but moderate CPU resources can be placed on a node with high GPU cores/memory but average CPU and RAM specs. Conversely, CPU-bound tasks that might benefit from GPU acceleration but don't require top-tier GPU resources can utilize nodes with high-end CPU, RAM, and modest GPU specs.

This design offers the benefit of tailored configurations for varied workloads, maximizing resource utilization and efficiency. However, the challenge is in workload placement. Proper tools and algorithms are essential to ensure that workloads are allocated to the most appropriate node configuration, ensuring optimal performance and avoiding resource wastage or underutilization.

5 Pods in the cluster

The dataset encompasses information pertaining to 8,152 pods, inclusive of configurations, pod phases, and timestamps indicating creation, scheduling, and deletion. Scheduling time data is conspicuously absent for 897 (11%) pods, which are currently in the pending phase. The comprehensive trace spans 149 days, 8 h, and 16 min, albeit the overwhelming majority (99.67%) of the records are concentrated post the 115th day. The inception date of the dataset remains unspecified. A total of 13.35% of the pods do not necessitate GPU support, contrasting with the remaining 86.65% that mandatorily require either a partial or multiple GPU allocations.

5.1 Scheduling metrics

The histograms shown in Fig. 4 illustrate the distribution of three parameters: interarrival time, scheduling delay, and creation time, after the removal of outliers and non-finite values. The interarrival time represents the intervals between the creation times of consecutive pods, the scheduling delay indicates the time elapsed

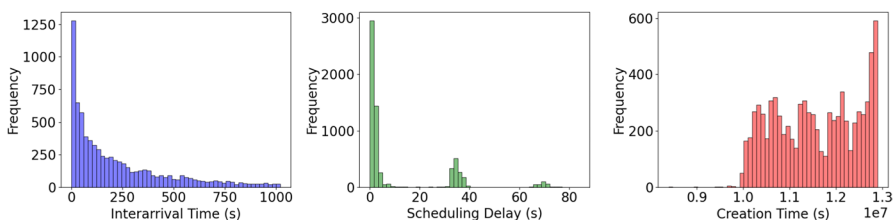


Fig. 4 Distribution of interarrival, scheduling delay, and creation times

between when a pod is scheduled to a node and when it was initially created, and the creation time marks when the pod is first recognized by Kubernetes. Outliers are removed using the Interquartile Range (IQR) method, which excludes values below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$, enhancing the precision of the analysis by minimizing the influence of extreme data points.

The average interarrival time is approximately 221.79 s, with a standard deviation of 239.35 s. 50% of the interarrival times (median) are below 133 s, and 75% are under 334 s, underscoring the prevalence of shorter interarrival times. However, the maximum recorded interarrival time is 1,021 s, highlighting the presence of longer interarrival times.

For scheduling delay, the average is around 11.22 s, with a standard deviation of 18.30 s. The median scheduling delay is only 2 s, with 75% of the delays being under 30 s, reinforcing the general efficiency of the scheduling system. Despite this, the maximum delay recorded stands at 84 s, emphasizing occasional instances of extended scheduling delay.

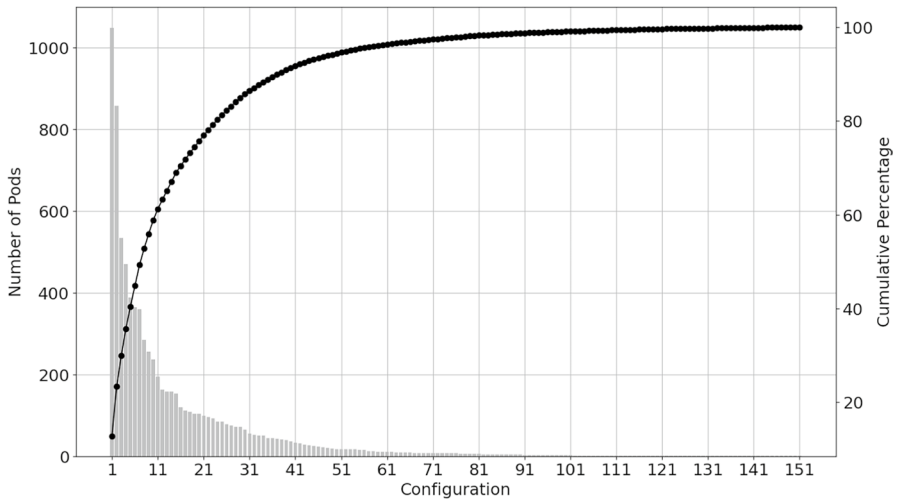
The pod creation times present a distinctly multi-modal distribution with notable activity spikes. While the early phase, around 8.4 to 10 million seconds, sees sparse pod creations, a pronounced increase emerges around 10 million seconds. This trend sees periodic peaks, most notably around 10.3 million and 12.7 million seconds. The crescendo of pod creation is observed in the final interval, near 12.9 million seconds, marking the highest concentration of creations. These fluctuations might indicate specific periods of heightened demand or system activity, suggesting that there could be external factors or scheduled tasks influencing the creation of pods during these peak times.

The outliers in the dataset which are excluded from the reported data above, exhibit significant deviations from the typical values, illuminating potential exceptional or anomalous situations within the scheduling and processing system. For interarrival time, the outliers, totaling 724, have a startling mean of approximately 15,544 s, with a maximum reaching an exceptional 1,410,436 s, indicating rare but considerable delays between task creations. In terms of scheduling delay, 858 outliers were identified with a mean of around 434 s, and a high value of 14,330 s, pointing towards instances of notable delays in scheduling.

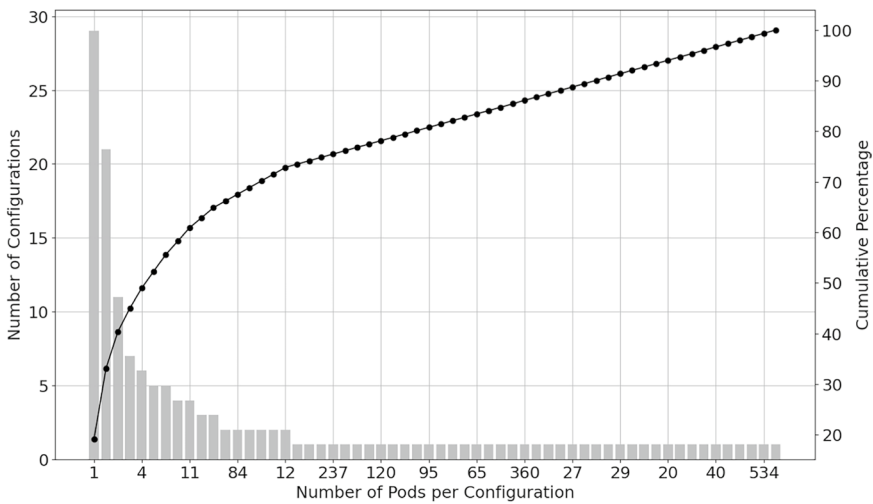
Lastly, the outlier pod creation times range from 0 s to approximately 7 million seconds. These times are either significantly earlier or noticeably later than the majority of the pod creation times in the primary dataset. This discrepancy might suggest distinct phases or specific events within the system, possibly related to testing, configuration, or other exceptional occurrences. These outlier pods could represent either an initial deployment preceding the main wave of pod creations or unique tasks that followed it.

5.2 Configurations

Figure 5 illustrates two Pareto diagrams that depict the distribution of pod configurations and the frequency of their appearances within the trace. A configuration refers to a unique set of resource requirements, including CPU, memory, and



(a) Pod counts per configuration.



(b) Number of configurations by appearance frequency.

Fig. 5 Distribution and frequency of pod configurations

GPU specifications. The total number of distinct pod configurations in the trace is 151. Figure 5a shows the number of pods associated with each configuration, revealing that a small subset of configurations is utilized by a large number of pods. Figure 5b indicates how many configurations appear once, twice, and so on, demonstrating that a significant number of configurations are used infrequently. These diagrams highlight that while there is a wide variety of configurations, the majority of pods employ a limited set of common configurations.

Table 2 Top ten configurations

CPU	RAM (GiB)	GPU Count	GPU	Pod Count	Cumulative (%)
3.15	5.47	1	0.81	1047	12.84
11.30	48	1	1	857	23.36
11.40	47	1	1	534	29.91
3.15	5.47	1	1	470	35.67
11.91	46	1	0.47	388	40.43
12.50	56	0	0	364	44.90
8	29.80	1	0.47	360	49.31
32	48	0	0	284	52.80
18.71	63	1	1	255	55.92
11.91	46	1	0.65	237	58.83

The majority of the pods, approximately 58.83%, are made up by the top ten configurations. Table 2 enumerates the ten most frequently occurring pod configurations in the trace. These configurations predominantly consist of setups with 1 GPU, with CPU values ranging mainly from 3.15 to 32, and RAM from about 5.47 GiB to 63 GiB. The variation in GPU requirements within these top configurations is also notable, fluctuating between 0.47 and 1.

In terms of frequency, 29 configurations appear only once in the trace, constituting 19.21% of the total. Configurations appearing less than ten times sum up to 93, accounting for 61.59% of the total, underscoring the diversity in configuration requirements. This pattern indicates a wide range of unique, task-specific requirements, emphasizing the need for flexibility and customization in resource allocation for optimal task performance and efficiency in diverse computational tasks. The data highlights the predominance of certain pod configurations and the extensive variety in configuration needs.

Figure 6 showcases a correlation heatmap of computing metrics, including CPU, GPU, RAM, and GPU Count. The most pronounced positive correlation is observed between CPU and RAM. Conversely, CPU and GPU exhibit a minor negative association, while GPU Count shows moderate correlations with both RAM and GPU. This heatmap is instrumental in understanding the intertwined nature of computational demands. When scheduling pods, these correlations suggest that pods with high CPU requirements are also likely to demand more RAM. However, increased CPU doesn't necessarily imply a higher GPU need. Additionally, when allocating multiple GPUs to a pod, there could be a simultaneous rise in both GPU units and RAM usage.

5.3 Quality of service

The distribution of pods across various QoS categories reveals that a significant proportion are categorized as LS, making up 57.01% of the total. This is closely followed by BE pods, which constitute 41.68%. The remaining pods fall under the *Burstable* category at 1.23% and a marginal fraction under *Guaranteed* at 0.09%.

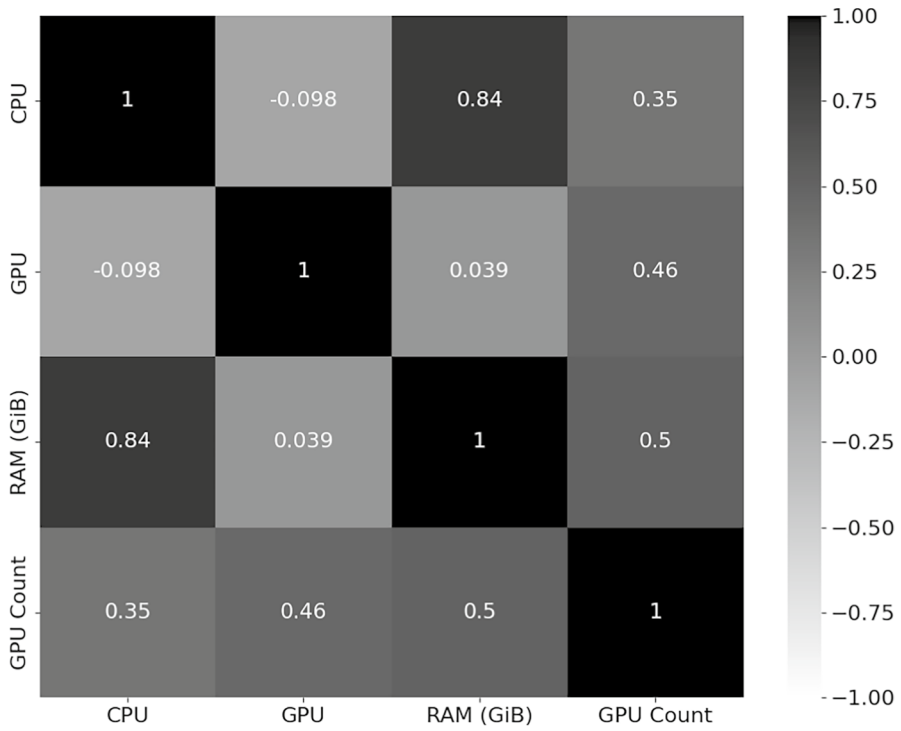


Fig. 6 Correlation of pod configurations

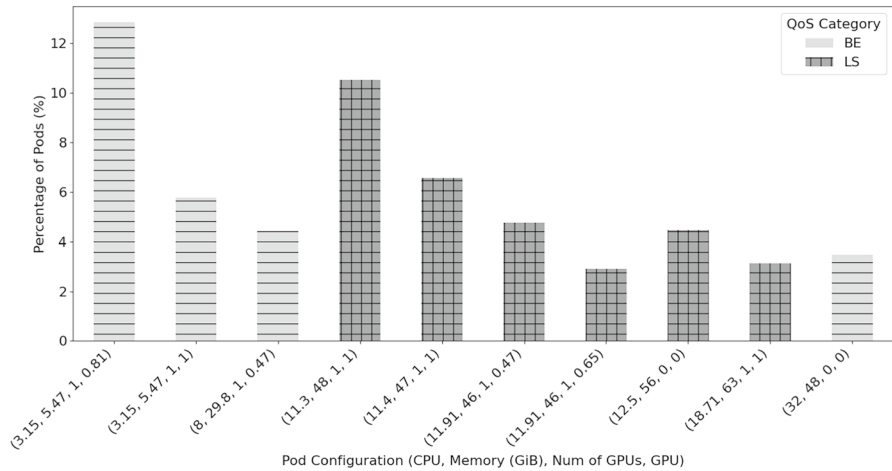


Fig. 7 Distribution of QoS categories for top ten pod configurations

Such a distribution suggests that while there's a dominance of LS tasks, a significant portion of the workload is also designed to operate under best-effort conditions, potentially leveraging any available excess capacity.

The ten configurations highlighted in Fig. 7 are the most recurrent in the trace, as previously detailed in Table 2. The most frequent configuration, with a CPU allocation of (3.15 CPU, 5.47 GiB of RAM, 1 GPU, and 0.81 GPU), is exclusively associated with the BE category. Interestingly, while several configurations show exclusive association with either LS or BE QoS classes, some configurations, like the (8 CPU, 29.8 GiB of RAM, 1 GPU, 0.47 GPU), cater to both BE and a negligible fraction of LS. These patterns highlight a relationship between specific resource configurations and their associated QoS classes, suggesting that certain configurations might be better suited for specific operational requirements. Another intriguing observation is the distinction between GPU and non-GPU pods within these configurations. For example, the configuration (12.5 CPU, 56 GiB of RAM) does not utilize any GPUs, yet it is classified under LS, indicating the presence of latency-sensitive tasks that do not necessarily require GPU acceleration.

5.4 Lifecycle

Of the total pods in the trace, a significant majority (63.70%) are actively in the *Running* phase, equating to 5,193 pods. However, a concerning 22.94% or 1,870 pods have *Failed*. Additionally, 11.00% (897 pods) are in a *Pending* state, awaiting necessary resources or facing constraints, while a mere 2.36% or 192 pods have successfully *Succeeded*, completing their designated tasks. The data underscores the system's operational capacity but also points to areas requiring further investigation and optimization, especially given the substantial number of failed pods.

In the trace, deletion times are evident across all pod phases, including *Pending* and *Failed*. Pods in the *Pending* phase, which await resource allocation or scheduling, may be deleted due to exceeding wait thresholds, system limitations, or interventions to relieve resource constraints. *Failed* pods, representing unsuccessful operations, might be removed following failure analyses, resource reallocation efforts, or to ensure a streamlined operational environment.

Our correlation analysis reveals minimal interplay between different pod phases, underscoring that the behavior of one pod doesn't significantly influence others within this period. Additionally, we observed a negligible correlation between pod configuration and pod phase which suggests that the system neither distinctly favors large nor small pods. However, this lack of preference cannot be conclusively determined from the data at hand.

Figure 8 illustrates the count of pods in each phase within the system at any given moment. The graph predominantly features pods in the *Running* phase, indicating an actively engaged system. As we progress along the timeline, there's a steady rise in the *Running* pods, hinting at an increasing system workload or task deployment. Additionally, sporadic spikes in the *Running* phase suggest peak traffic times, offering schedulers insights for potential optimizations. The *Failed* pods appear

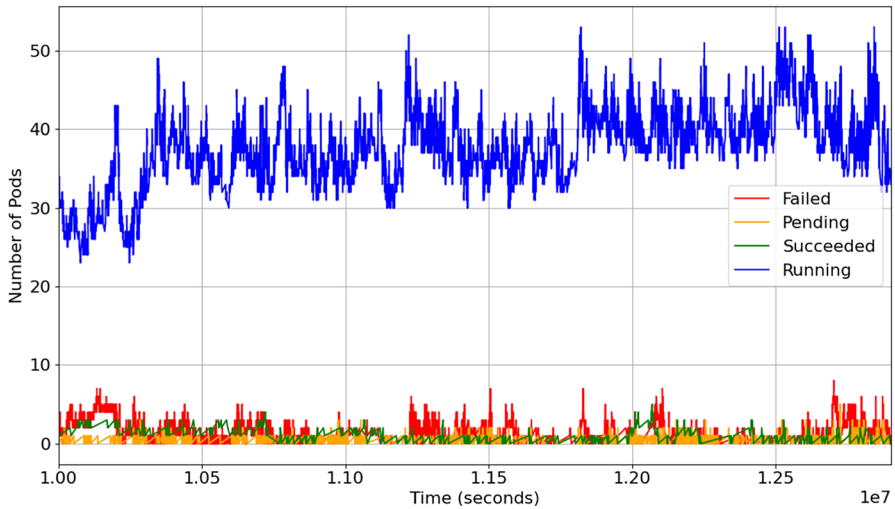


Fig. 8 Temporal analysis of pod phases

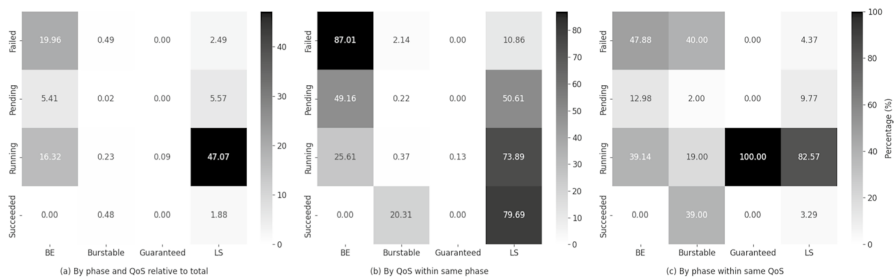


Fig. 9 Pod distributions across phases and QoS classes

intermittently, without a clear trend and *Succeeded* pods appear in small numbers occasionally which may indicate execution of specific tasks.

Figure 9a illustrates the distribution of pods among different phases and QoS classes. It's clear that the majority of the pods are in the *Running* and *Failed* phases, predominantly within the LS and BE classes. Notably, 47.07% of the total pods are in the *Running* phase under the LS class. The next significant group is the pods in the *Failed* phase within the BE class, making up 19.96% of the total. This indicates a system bias towards LS pods, which aligns with expectations.

In Fig. 9b, the distribution of pods within each phase across the QoS classes is depicted. Among the *Failed* pods, a significant 87.01% belong to the BE class, followed by 10.86% in the LS class. Interestingly, there are no *Failed* pods in the *Guaranteed* category, which is consistent with expectations. A mere 2.14% of the *Failed* pods are from the *Burstable* class. When we compare these numbers to the overall distribution of pods in each class, as presented in Fig. 9c, it's evident that 47.88% of BE class pods, 40% of *Burstable* class pods, and 4.37% of LS class pods have

experienced failure. This observation reinforces the earlier inference that the system is optimized to favor LS pods, which also constitute the bulk of the system's pods.

In the *Pending* phase, BE and LS pods are nearly evenly distributed at 5.41% and 5.57%, respectively, and contribute almost equally to the *Pending* category (49.16% BE and 50.61% LS). This represents 12.98% of all BE pods and 9.77% of all LS pods. Although this distribution seems to contradict the system's apparent preference for LS pods, the *Running* phase shows a dominant 73.89% of LS pods, suggesting that scheduling policies like affinity or resource availability might influence the *Pending* distribution.

A mere fraction of all pods conclude in the *Succeeded* phase within the Burstable (0.48%) and LS (1.88%) classes. While this constitutes 39% of all Burstable pods, it represents just 3.29% of LS pods, indicating a tendency for certain specific pods to finish in this phase. Figure 10 presents the duration of these phases, revealing variations from other phases within the same QoS classes, further supporting the notion that these pods may be distinct.

In the analysis illustrated in Fig. 10, we examined the durations of pods from when they are scheduled to when they are deleted, aiming to identify patterns related to pod phases and QoS classes. We excluded the *Pending* phase from this analysis as the pods in this phase are yet to be scheduled, and therefore, do not have a measurable duration.

In the category of failed pods, BE pods exhibited an average lifespan of roughly 252 s, just over 4 min, with a median duration of 194 s or less and a maximum duration of 971 s. This brief duration for failed BE pods implies potential immediate issues with resources or configurations, hindering their ability to run for longer periods. In the Burstable class, the average duration was around 1,210 s (20 min), with half of the pods failing within the first 255 s and the longest-running pod lasting for 6,568 s. For the LS class, the average duration before termination was approximately 1,632 s (27 min), with the longest duration reaching 6,090 s.

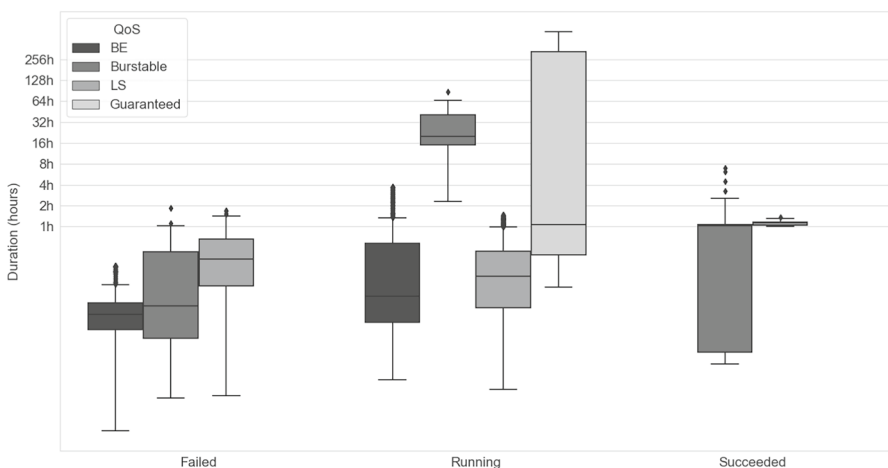


Fig. 10 Distribution of pod duration for each pod phase and QoS

For the actively running pods, the BE class comprised 1,123 instances, averaging a runtime of approximately 1,850 s, or around 31 min, with the longest duration being 13,341 s. The Burstable class had 16 pods, exhibiting notably lengthy average durations of around 111,302 s, or roughly 30.9 h, and a maximum duration of 315,378 s. Despite having only 7 pods, the Guaranteed QoS class demonstrated an exceptionally high average duration of 674,783 s, approximately 7.8 days. The LS class, with 3,393 running instances, had an average duration of 1,121 s, or 18.7 min, and a maximum duration of 5,300 s.

Comparing these durations to those of the failed phase, it's evident that the running durations are significantly longer across all classes. BE pods in the failed phase had an average duration of just over 4 min, substantially shorter than the 31 min for running BE pods. The Burstable and LS classes in the failed phase had average durations of 20 min and 27 min respectively, which are also much shorter than their counterparts in the running phase. This stark contrast in durations between the running and failed phases across all classes underscores the challenges and issues leading to pod failures, resulting in their significantly shorter lifespans.

Concluding with the pods that successfully completed their tasks, the Burstable class had 32 pods with an average duration of about 4,224 s or 70 min. The LS class had 111 pods, with an average duration of around 3,998 s. The longest-running pod in this category was timed at 4,840 s. The close alignment of average and median durations for succeeded LS pods indicates consistent performance and could mean that these pods are optimized for specific tasks with predictable execution times.

6 Conclusion

Our analysis of Alibaba's 2023 GPU cluster traces shows varied GPU node configurations and operational patterns. The cluster, consisting of 79.65% GPU and 20.35% non-GPU nodes, balances performance with a strong CPU-RAM correlation and enables flexible resource allocation due to a weak GPU-CPU/RAM correlation. Certain GPU models are matched with specific CPU and RAM ranges to efficiently cater to high-demand tasks. Covering nearly three months, most of the dataset's records appear in the last month. Of the pods, 86.65% require GPU support, while 13.35% do not. Pod creation times exhibit fluctuating demand and a multi-modal distribution. There are 151 unique pod configurations; the top ten make up 58.83% of the total, typically featuring a single GPU and varying CPU and RAM specs. Notably, 19.21% of configurations are singular, and 61.59% appear less than ten times, reflecting a wide range of requirements. Resource correlation analysis reveals a strong CPU-RAM relationship, a weak CPU-GPU link, and a moderate correlation between GPU count and both RAM and GPU, providing critical insights for improving resource allocation in cluster environments.

The pod distribution reveals a dominance of latency-sensitive tasks (57.01%), underscoring the system's emphasis on quick task execution, with a varied need for GPU acceleration. A substantial 41.68% of the workload operates under best-effort conditions, utilizing surplus capacity, yet challenges are apparent with 22.94% of pods failing and 11.00% pending, highlighting areas for optimization. The system

demonstrates an equitable approach to resource allocation, with the behavior of individual pods being largely independent of one another and no apparent bias towards pods of differing sizes. Future work should focus on developing efficient scheduling algorithms, improving resource allocation strategies, and further investigating the causes behind pod failures and pending states. Additionally, exploring the impact of different node and pod configurations on overall system performance will be crucial. This study stands as a beneficial resource for researchers and practitioners in the field, aiding in the advancement of GPU cluster management and operation.

Author Contributions Ahmad Siavashi led the research, data analysis, and manuscript preparation. Mahmoud Momtazpour, the advisor, contributed through critical review and insights, guiding the study's direction and refinement.

Funding Not applicable.

Data Availability No datasets were generated or analysed during the current study.

Declaration

Conflict of interest The authors declare no Conflict of interest.

References

1. Senjab K, Abbas S, Ahmed N, Khan AUR (2023) A survey of Kubernetes scheduling algorithms. *J Cloud Comput* 12(1):87
2. Shen W, Liu Z, Tan Y, Luo Z, Lei Z (2023) KubeGPU: efficient sharing and isolation mechanisms for GPU resource management in container cloud. *J Supercomput* 79(1):591–625
3. Ishak H, Makhlof SA, Belalem G (2022) KubeSC-RTP: Smart scheduler for Kubernetes platform on CPU-GPU heterogeneous systems. *Concurr Comput Pract Exp* 34(21):e7108
4. Hijma P, Heldens S, Sclocco A, Van Werkhoven B, Bal HE (2023) Optimization techniques for GPU programming. *ACM Comput Surv* 55(11):1–81
5. Weng Q, Yang L, Yu Y, Wang W, Tang X, Yang G, Zhang L (2023) Beware of fragmentation: scheduling GPU-sharing workloads with fragmentation Gradient Descent. In: 2023 USENIX Annual Technical Conference (USENIX ATC 23), pp. 995–1008
6. Weng Q, Xiao W, Yu Y, Wang W, Wang C, He J, Li Y, Zhang L, Lin W, Ding Y (2022) MLaaS in the wild: workload analysis and scheduling in large-scale heterogeneous GPU clusters. In: 19th USENIX symposium on networked systems design and implementation (NSDI 22), pp. 945–960
7. Alibaba Cluster Trace GPU v2023, Accessed: 2023-10-21. <https://github.com/alibaba/clusterdata/tree/master/cluster-trace-gpu-v2023>
8. Shan Y, Huang Y, Chen Y, Zhang Y (2018) LegoOS: a disseminated, distributed OS for hardware resource disaggregation. In: 13th USENIX symposium on operating systems design and implementation (OSDI 18), pp. 69–87
9. Liu Q, Yu Z (2018) The elasticity and plasticity in semi-containerized co-locating cloud workload: a view from Alibaba trace. In: Proceedings of the ACM symposium on cloud computing, pp. 347–360
10. Shen H, Li C (2018) Zeno: a straggler diagnosis system for distributed computing using machine learning. In: High performance computing: 33rd international conference, ISC high performance 2018, Frankfurt, Germany, June 24–28, 2018, Proceedings 33. Springer, pp. 144–162

11. Cheng Y, Chai Z, Anwar A (2018) Characterizing co-located datacenter workloads: an Alibaba case study. In: Proceedings of the 9th Asia-Pacific Workshop on Systems, pp. 1–3
12. Lu C, Ye K, Xu G, Xu C-Z, Bai T (2017) Imbalance in the cloud: an analysis on Alibaba cluster trace. In: 2017 IEEE international conference on big data (big data). IEEE, pp. 2884–2892
13. Jiang C, Han G, Lin J, Jia G, Shi W, Wan J (2019) Characteristics of co-allocated online services and batch jobs in internet data centers: a case study from Alibaba cloud, IEEE Access 7 22495–22508, publisher: IEEE
14. Guo J, Chang Z, Wang S, Ding H, Feng Y, Mao L, Bao Y (2019) Who limits the resource efficiency of my datacenter: an analysis of Alibaba datacenter traces. In: Proceedings of the International Symposium on Quality of Service, pp. 1–10
15. Li F, Hu B (2019) DeepJS: job scheduling based on deep reinforcement learning in cloud data center. In: Proceedings of the 4th International Conference on Big Data and Computing, pp. 48–53
16. Tian H, Zheng Y, Wang W (2019) Characterizing and synthesizing task dependencies of data-parallel jobs in Alibaba cloud. In: Proceedings of the ACM Symposium on Cloud Computing, pp. 139–151
17. Wu H, Zhang W, Xu Y, Xiang H, Huang T, Ding H, Zhang Z (2019) Aladdin: optimized maximum flow management for shared production clusters. In: IEEE international parallel and distributed processing symposium (IPDPS). IEEE, pp. 696–707
18. Luo S, Xu H, Lu C, Ye K, Xu G, Zhang L, Ding Y, He J, Xu C (2021) Characterizing microservice dependency and performance: Alibaba trace analysis. In: Proceedings of the ACM Symposium on Cloud Computing, pp. 412–426
19. Detti A, Funari L, Petrucci L (2023) μ bench: an open-source factory of benchmark microservice applications. IEEE Trans Parallel Distrib Syst 34(3):968–980
20. Wang K, Li Y, Wang C, Jia T, Chow K, Wen Y, Dou Y, Xu G, Hou C, Yao J others (2022) Characterizing job microarchitectural profiles at scale: dataset and analysis. In: Proceedings of the 51st International Conference on Parallel Processing, pp. 1–11
21. Pod Lifecycle, Accessed: 2024-03-28. <https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/>
22. Pod Quality of Service Classes, Accessed: 2024-03-28. <https://kubernetes.io/docs/concepts/workloads/pods/pod-qos/>
23. NVIDIA T4 Tensor Core GPU for AI Inference | NVIDIA Data Center, Accessed: 2023-09-26. <https://www.nvidia.com/en-us/data-center/tesla-t4/>
24. A10 Tensor Core GPU | NVIDIA, Accessed: 2023-09-26. <https://www.nvidia.com/en-us/data-center/products/a10-gpu/>
25. NVIDIA V100, Accessed: 2023-09-26. <https://www.nvidia.com/en-us/data-center/v100/>
26. Tesla P100 Data Center Accelerator | NVIDIA, Accessed: 2023-09-26. <https://www.nvidia.com/en-us/data-center/tesla-p100/>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.